



Java Object- Oriented Programming (OOP)

AN INTRODUCTION TO OOP CONCEPTS

Introduction to OOP

- ▶ Java is an object-oriented programming (OOP) language that organizes code into objects and classes. The four main principles of OOP are:
- ▶ 1. Encapsulation
- ▶ 2. Inheritance
- ▶ 3. Polymorphism
- ▶ 4. Abstraction

Inheritance

- ▶ Inheritance allows one class to inherit the fields and methods of another class, enabling code reuse and hierarchy structure.
- ▶ Example 1: A subclass inheriting properties from a superclass.
- ▶ Example 2: Overriding methods in a subclass.
- ▶ Example 3: Using 'super' keyword to access parent class methods.

Polymorphism

- ▶ Polymorphism allows one interface to be used for a general class of actions. The specific action is determined by the exact nature of the situation.
- ▶ Example 1: Method overloading (multiple methods with same name but different parameters).
- ▶ Example 2: Method overriding in inheritance (subclass overrides a parent class method).
- ▶ Example 3: Using interfaces to achieve polymorphism.

Encapsulation

- ▶ Encapsulation refers to the bundling of data (variables) and methods (functions) that operate on the data within a single unit or class. This ensures data security and hides implementation details from outside access.
- ▶ Example 1: Private fields with public getters and setters.
- ▶ Example 2: Class with encapsulated attributes.
- ▶ Example 3: Encapsulation using access modifiers (public, private).

Abstraction

- ▶ Abstraction simplifies complex reality by modeling classes based on essential qualities, leaving out the irrelevant details.
- ▶ Example 1: Abstract class with abstract and concrete methods.
- ▶ Example 2: Interface defining methods to be implemented.
- ▶ Example 3: Hiding implementation details using abstract methods.



Java Swing

AN OVERVIEW OF JAVA SWING

What is Java Swing?

- ▶ Java Swing is a part of Java Foundation Classes (JFC) used to create window-based applications. It provides a rich set of components such as buttons, checkboxes, tables, etc., to build GUI applications.
- ▶ It is built on top of AWT (Abstract Window Toolkit) and provides more sophisticated components.

Example 1: Creating a Simple Window

- This example shows how to create a basic JFrame window in Swing.

```
import javax.swing.JFrame;
public class SimpleWindow {
    public static void main(String[] args) {
        JFrame frame = new JFrame('Simple Window');
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Example 2: Adding Components to JFrame

- In this example, we add a button and a label to the JFrame.

```
import javax.swing.*;

public class AddComponents {
    public static void main(String[] args) {
        JFrame frame = new JFrame('Add Components');
        JButton button = new JButton('Click Me');
        JLabel label = new JLabel('Hello, Swing!');
        frame.add(button);
        frame.add(label);
        frame.setLayout(new FlowLayout());
        frame.setSize(300, 200);
        frame.setVisible(true);
    }
}
```

Example 3: Handling Events in Swing

- ▶ This example shows how to handle button click events using an ActionListener.

```
import javax.swing.*;
import java.awt.event.*;

public class EventHandling {
    public static void main(String[] args) {
        JFrame frame = new JFrame('Event Handling');
        JButton button = new JButton('Click Me');
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                System.out.println('Button Clicked!');
            }
        });
        frame.add(button);
        frame.setSize(300, 200);
        frame.setVisible(true);
    }
}
```

Benefits of Java Swing

- ▶ 1. Lightweight components: Swing provides lightweight components that can be customized.
- ▶ 2. Platform Independent: Java Swing applications are platform-independent.
- ▶ 3. Rich set of components: Provides advanced components such as tables, trees, scroll panes, etc.